Lorenz System

> **Note**
>
> Learning and testing how **Neural ODEs** work by modeling a classical **three-variable differential equation system**: the **Lorenz Attractor**.

Explaining the Lorenz System

It is an inherently **chaotic system**, highly sensitive to initial conditions, making it difficult to solve analytically.

Differential Equations

The Lorenz system is a system of 3 ordinary differential equations:

$$\frac{dx}{dt} = \sigma(y - x)$$

$$\frac{dy}{dt} = x(\rho - z) - y$$

$$\frac{dz}{dt} = xy - \beta z$$

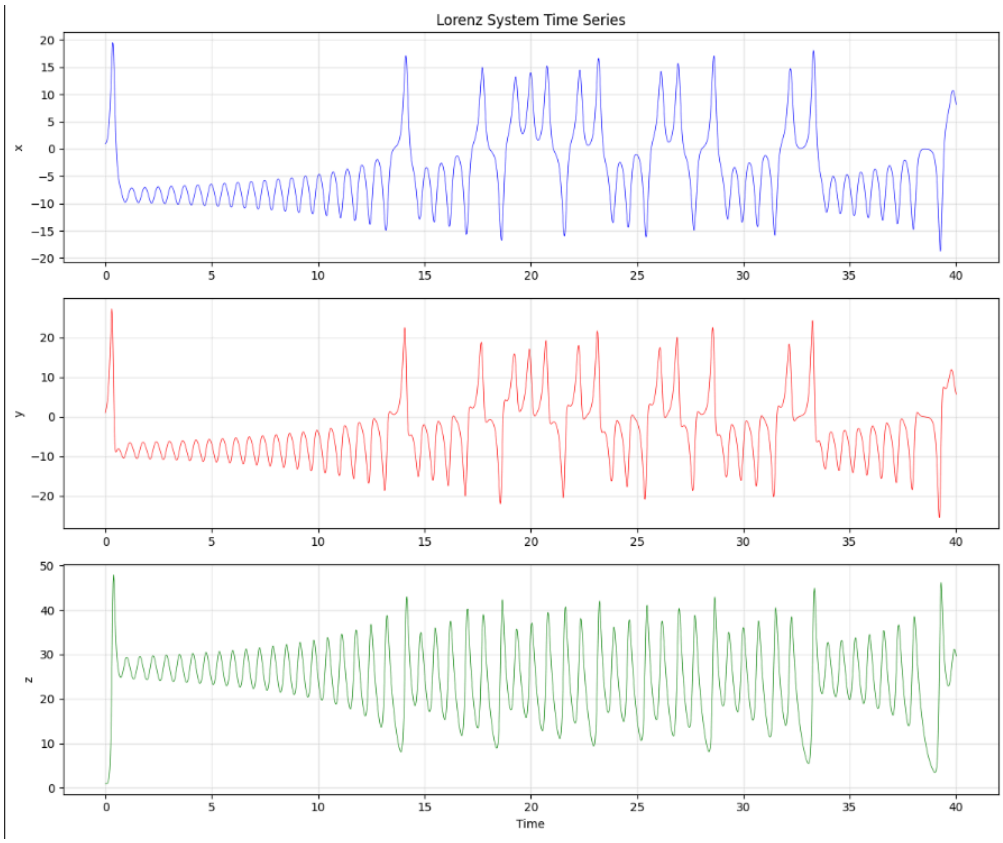
Where:

- σ → Prandtl number
- ρ → Rayleigh number
- β → geometric factor

These parameters control the **chaotic behavior** of the system.

> **Info**
>
> Unlike simple cyclic systems, the Lorenz system produces **non-repeating, butterfly-shaped trajectories**, known as a **strange attractor**.

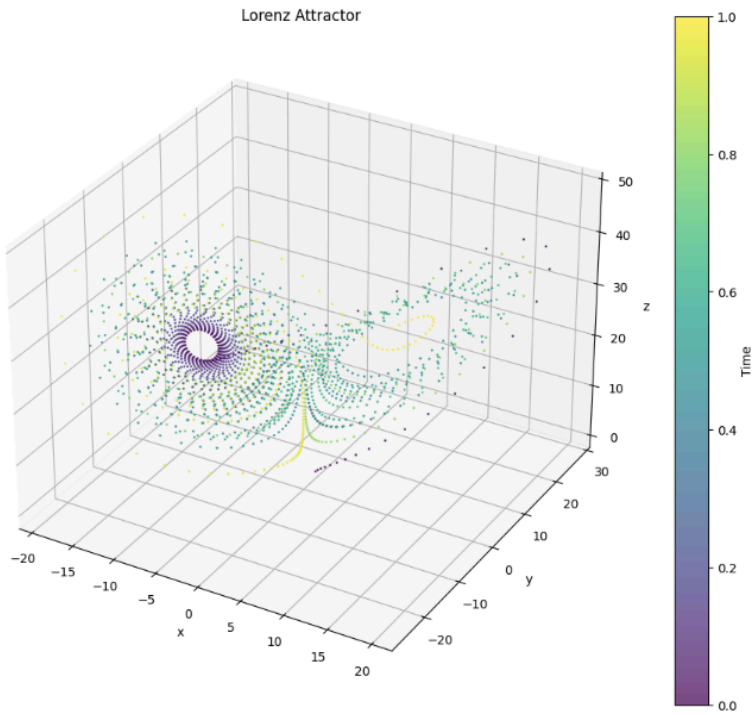
Lorenz Time Series Data



✓ **Observation**

- Each variable (x, y, z) evolves **non-linearly over time**
- No clear periodicity → indicates **chaos**
- Small variations grow rapidly → **sensitive dependence on initial conditions**

3D Phase Space (Lorenz Attractor)



✓ **Observation**

- The trajectory forms a **butterfly-shaped attractor**
- The system never settles → always evolving
- Motion switches between two lobes → **deterministic chaos**

Applying Neural ODE

Instead of explicitly defining equations, we use a **Neural ODE** to *learn the dynamics*.

What we are doing:

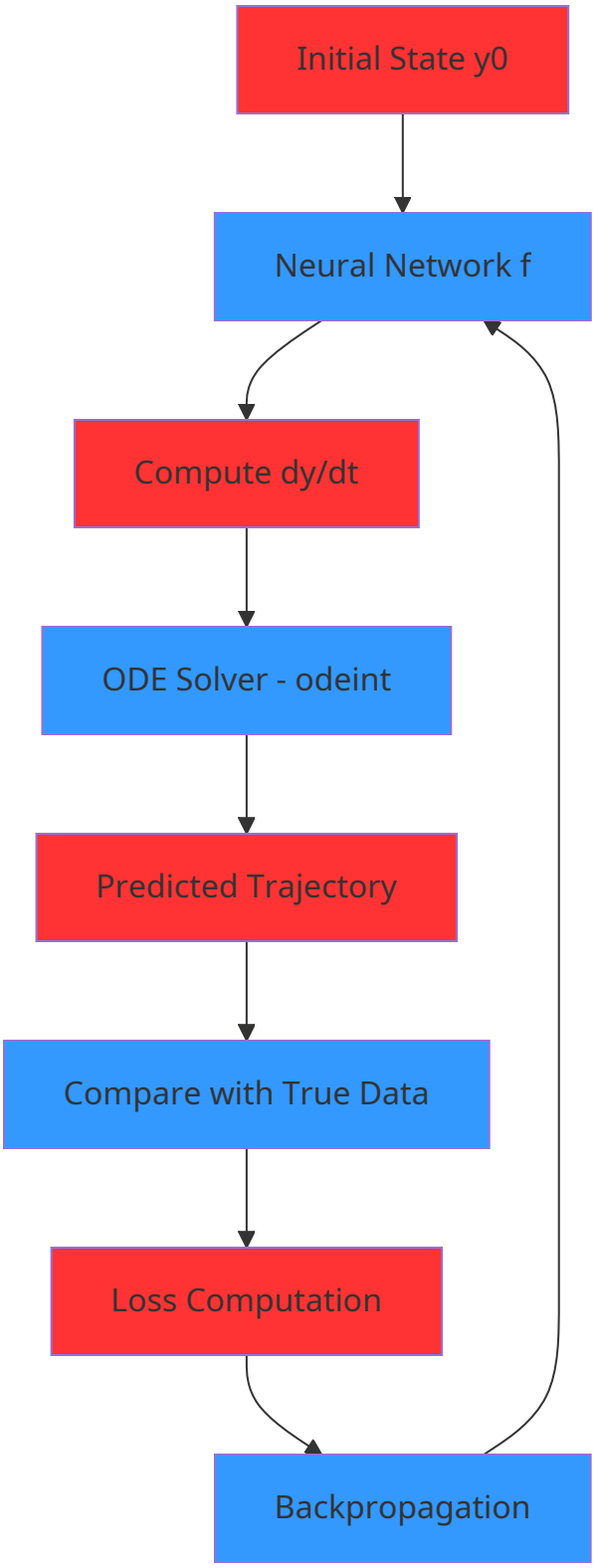
- Treat system as **continuous-time function**
- Replace analytical derivative with **neural network**
- Learn:

$$\frac{dy}{dt} = f_\theta(y)$$

Tip

The neural network approximates the **vector field** of the system.

Workflow (Neural ODE)



✓ **Insight**

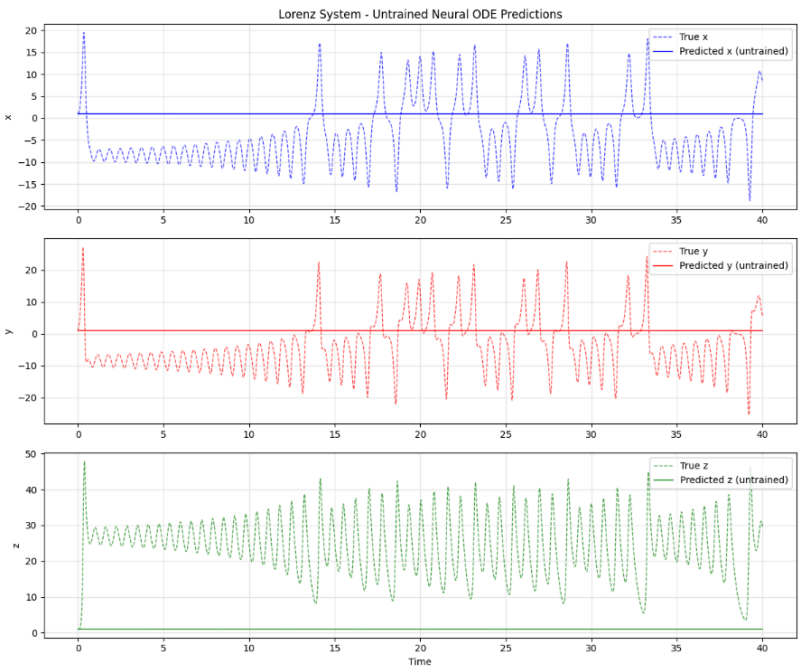
The model **learns system dynamics implicitly**, without knowing equations.

Model Architecture

- Input: (x, y, z)
- Hidden layers: Fully connected + **SiLU activation**
- Output: $(dx/dt, dy/dt, dz/dt)$

> **Info**
>
> Xavier initialization ensures **stable gradient flow**.

Test on Untrained Model



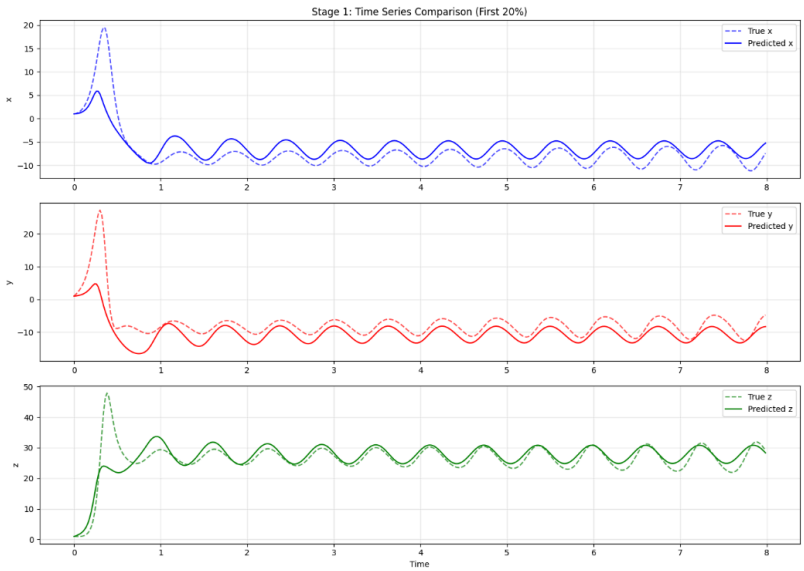
> **Observation**
>
> - Predictions are **random and unstable**
> - No resemblance to Lorenz dynamics
> - Confirms model must **learn structure from data**

Stage 1 — 20% Training

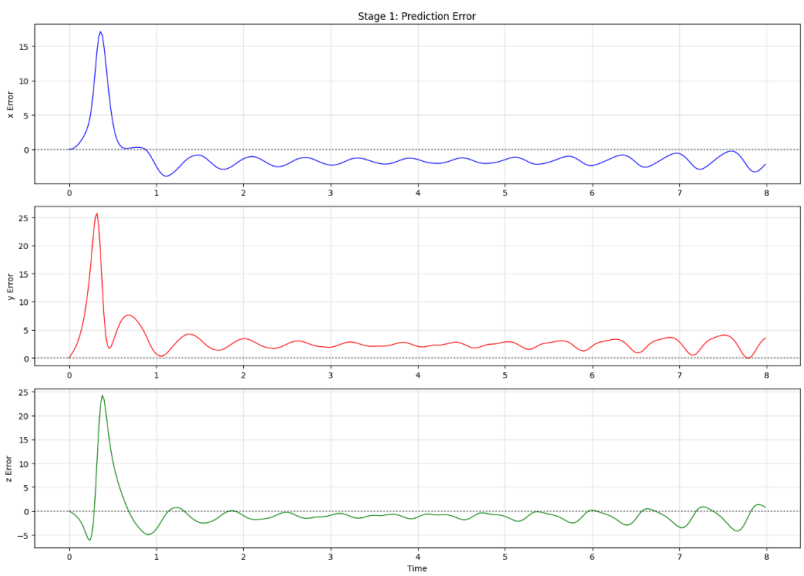
Params

- Trained on **first 20% of time series**
- 1000 epochs

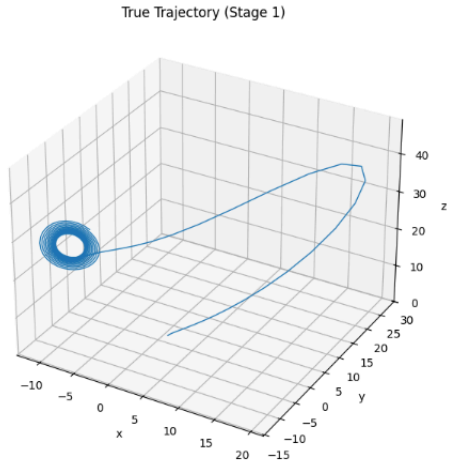
Time Series

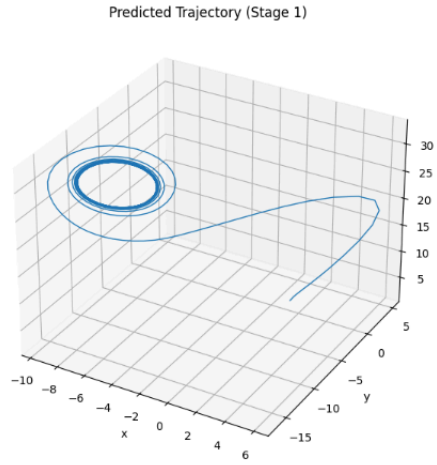


Error Plot



3D Phase Plot





✓ Observation

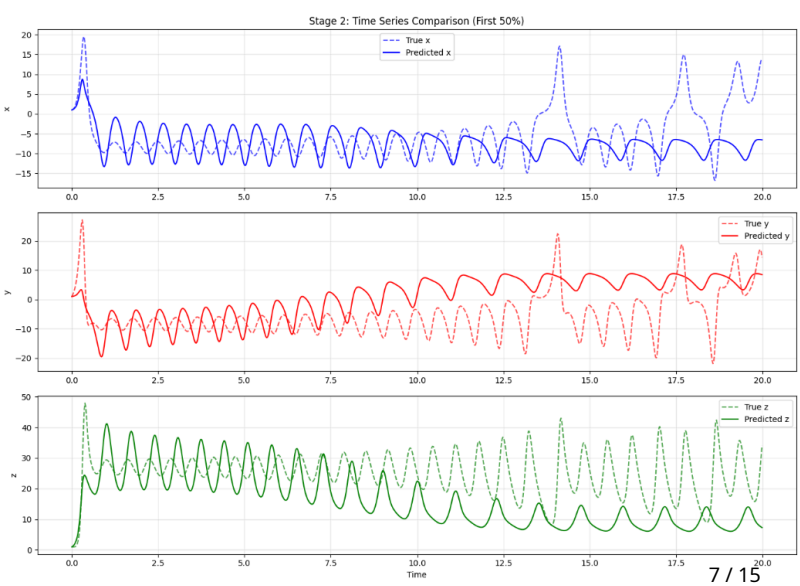
- Partial learning of trend
- Still deviates heavily
- Attractor shape not fully formed
- High prediction error

Stage 2 — 50% Training

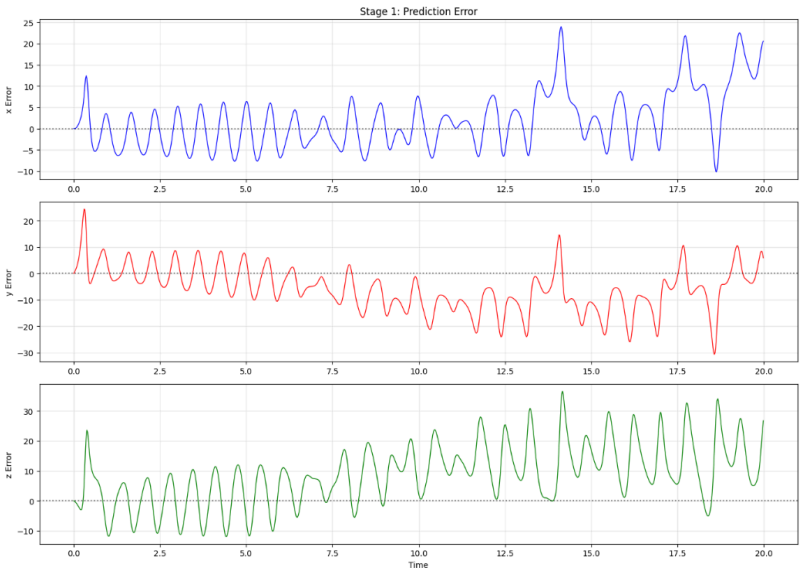
Params

- Trained on **first 50% of data**
- 1000 epochs

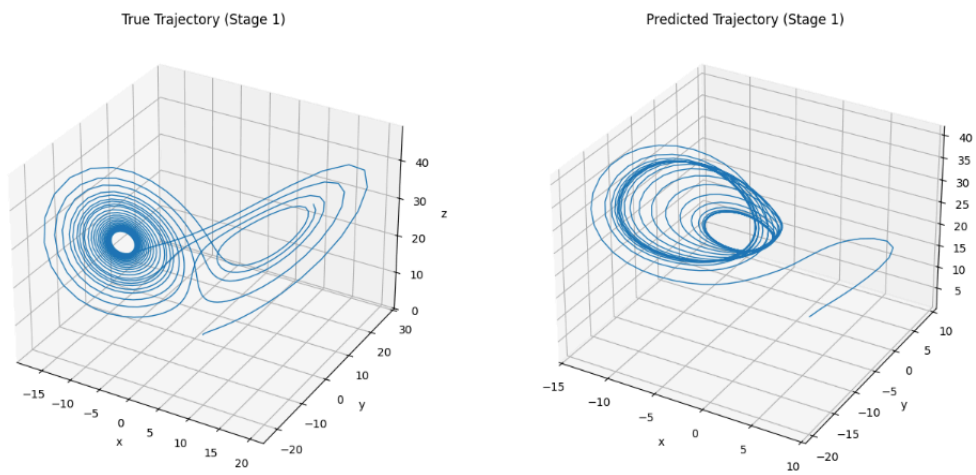
Time Series



Error Plot



3D Phase Plot



✓ Observation

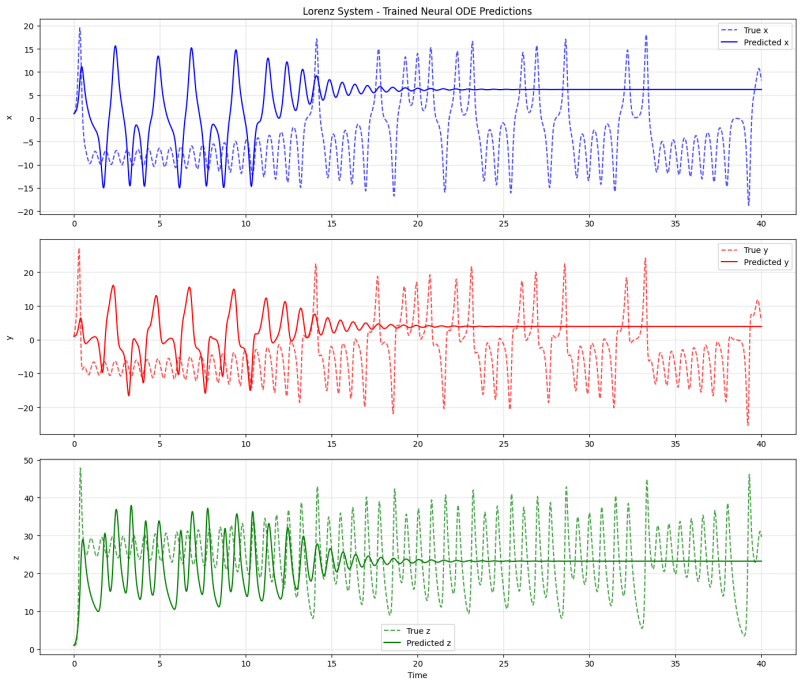
- Much closer to true dynamics
- Begins forming **butterfly structure**
- Error significantly reduced
- Still diverges over long horizon

Stage 3 — Full Training

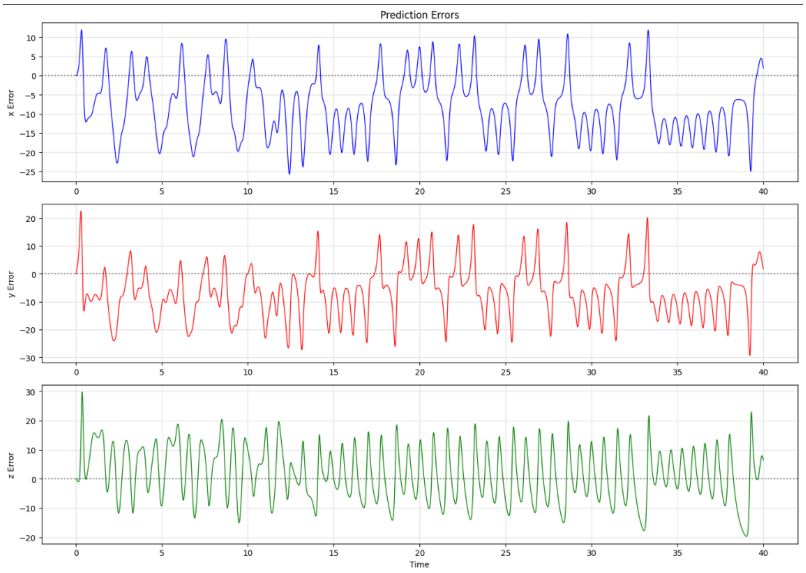
Params

- Trained on **entire dataset**
- 1000 epochs

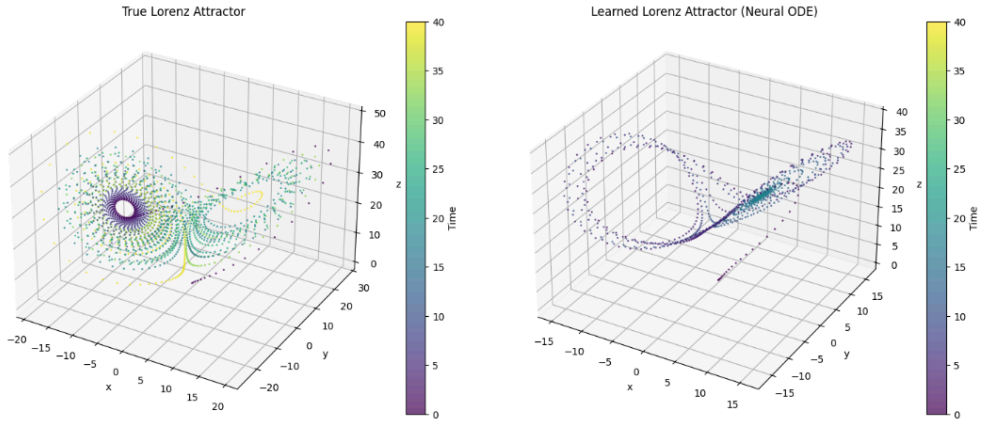
Time Series



Error Plot



3D Phase Plot



✓ **Observation**

- Low short-term error
- Divergence remains (chaos effect)
- Still able to learn a chaoitic system

Training Behavior (Windowed Learning)

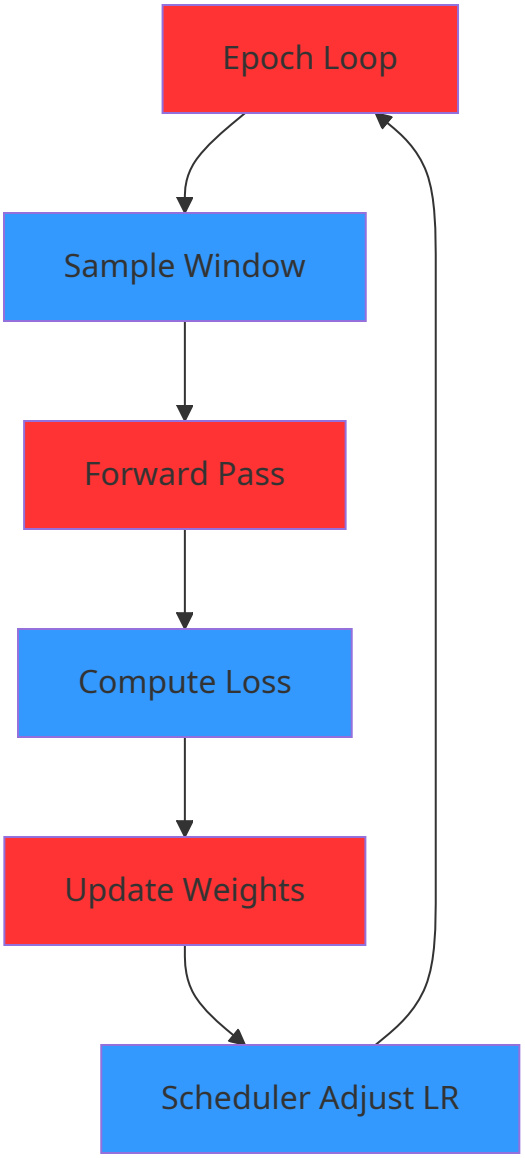
The model is trained using **random time windows**:

```
start_idx = np.random.randint(0, len(t_stage1) - window_size)

t_batch = t_stage1[start_idx:start_idx + window_size]
y_batch = y_stage1[start_idx:start_idx + window_size]

y0_batch = y_batch[0]
```

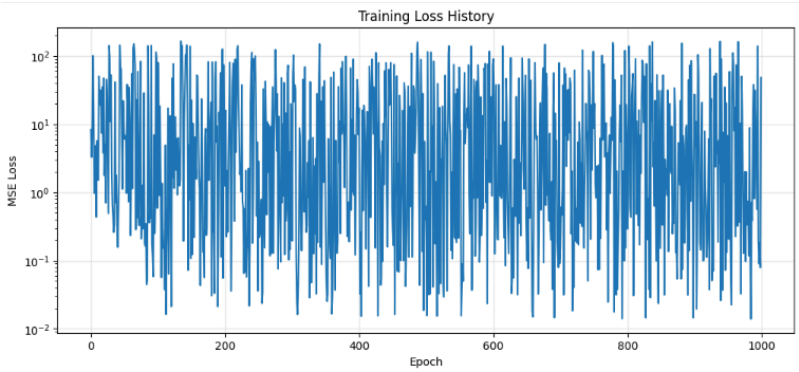
Training Loop



✓ **Observation**

- Windowing improves **generalization**
- Avoids overfitting full trajectory
- Helps learn **local dynamics efficiently**

Loss Curve

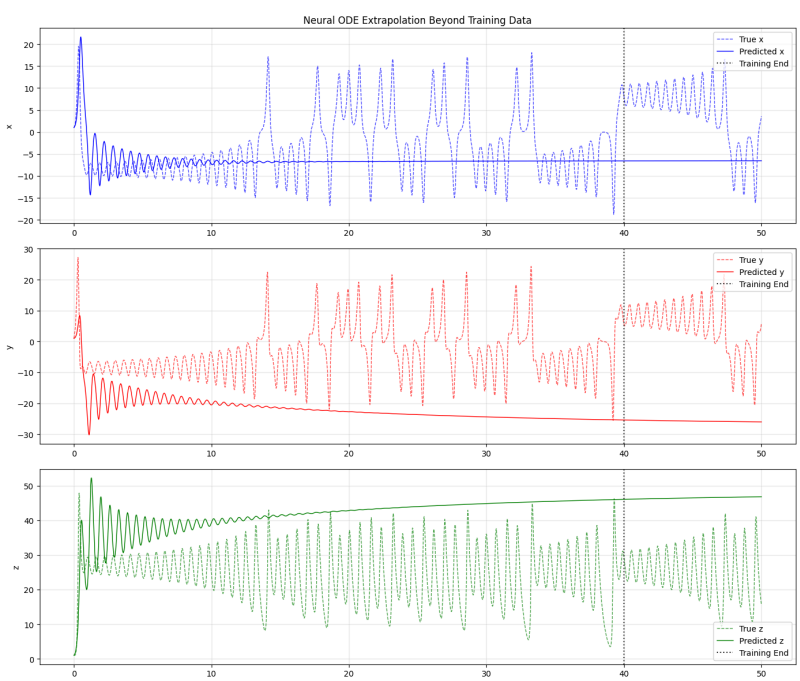


✓ **Observation**

- Loss decreases , but varies primaritly due to windowed training approach
- Fluctuations due to chaos
- Scheduler stabilizes training

Extrapolation Test

- Training: $(t = 0 \to 40)$
- Testing: $(t = 40 \to 50)$

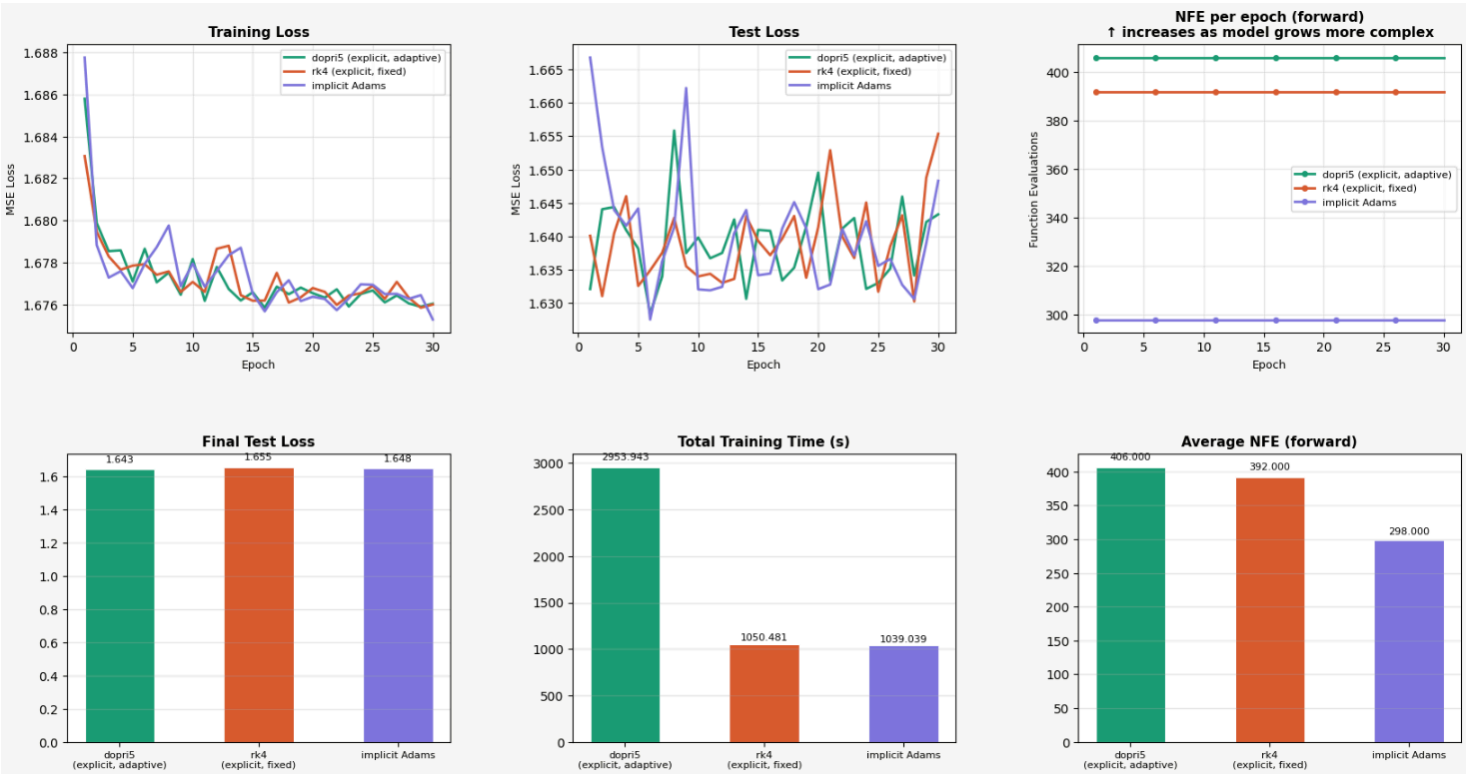


✓ **Observations**

- Model performs well **within initial raining range**
- Beyond range:
  - Divergence Occurs
  - Still follows **general structure**
- Shows Neural ODE captures **underlying dynamics**, but chaos limits long-term prediction

---

Implicit vs Explicit Solvers



ⓘ **Info**

We compare three ODE solvers used in Neural ODE training:

- **dopri5** → Explicit (adaptive step)
- **rk4** → Explicit (fixed step)
- **implicit_adams** → Implicit method

Performance Comparison

Solver Comparison

Solver	Type	Test Loss	Avg NFE	Time (s)
dopri5	Explicit	1.64329	406.0	2953.9
rk4	Explicit	1.65536	392.0	1050.5
implicit_adams	Implicit	1.64831	298.0	1039.0

Key Observations

✓ Accuracy

- **dopri5** → best (lowest loss)
- **implicit_adams** → close second
- **rk4** → worst

✓ Efficiency (NFE)

- **implicit_adams** → lowest (~298)
- Explicit methods → higher (~400)

✓ Training Time

- **implicit_adams ≈ rk4 (~1040s)** → fastest
- **dopri5 (~2950s)** → slow

Accuracy vs Efficiency Trade-off

Explicit (dopri5, rk4)

- Better accuracy
- More NFEs → slower

Implicit (implicit_adams)

- Fewer NFEs → faster
- Slightly lower accuracy

Conclusion

Note

Neural ODEs can:

- Learn **continuous-time dynamics**
- Approximate unknown systems
- Handle chaotic systems like Lorenz

But:

- Long-term prediction is limited due to **chaotic sensitivity**
- Best suited for **short-term accurate forecasting + system understanding**

Code References

See implementation of lorenz: `neural_ode.ipynb`
See implementation of Implicit vs Explicit : `node_lorenz.ipynb`